**UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO**

Facultad de Ciencias 2027-1

*Introducción a Ciencias de la Computación*

# REPORTE — PRÁCTICA 4

Dándole memoria a un programa

Nombre: López Pérez Luis Ángel

Número de cuenta: 324243174

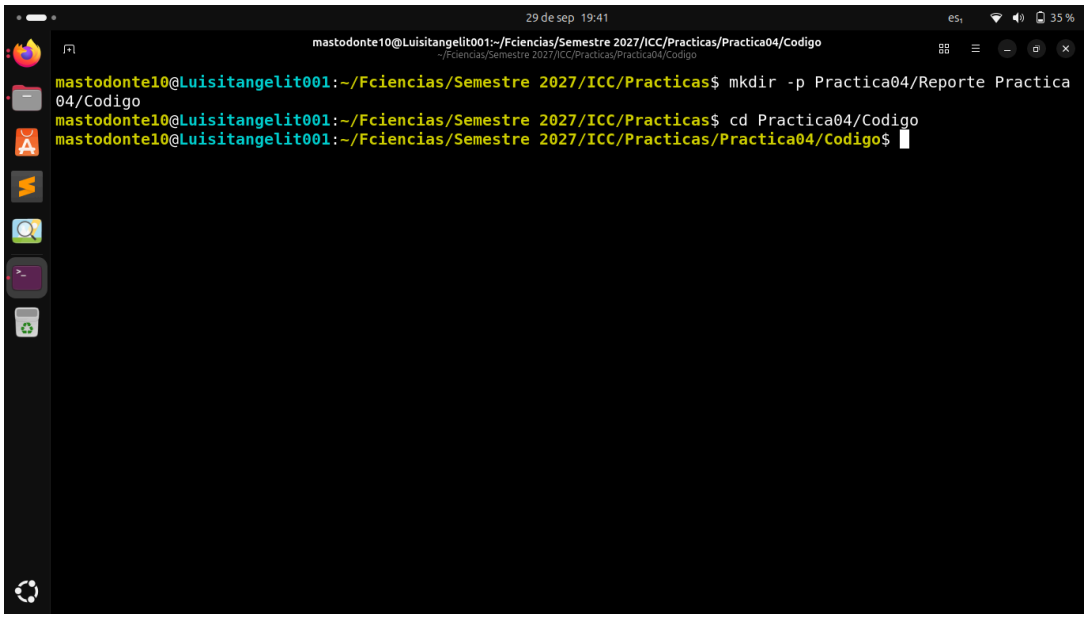
Sistema operativo: Linux (Ubuntu)

**Fecha: 30 / 09 / 2026**



# Parte 0 Preparación del directorio de la práctica

Antes de empezar se creó la estructura de directorios que pide la práctica. Desde ~/Fciencias/Semestre 2027/ICC/Practicas se ejecutó mkdir -p Practica04/Reporte Practica04/Codigo, con lo que se crearon de una sola vez la carpeta Practica04 y sus dos subcarpetas: Reporte (para el PDF) y Codigo (para los cuatro archivos Java). Después se entró a Codigo con cd.



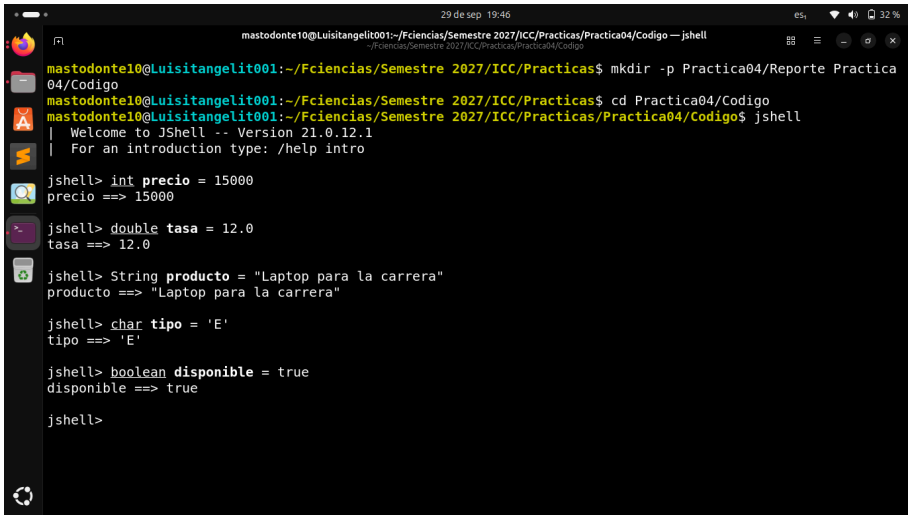

# Parte 1: Variables

## 1.1 Variables en JShell

Se abrió JShell con el comando jshell y se trabajaron los cinco ejercicios de la actividad.

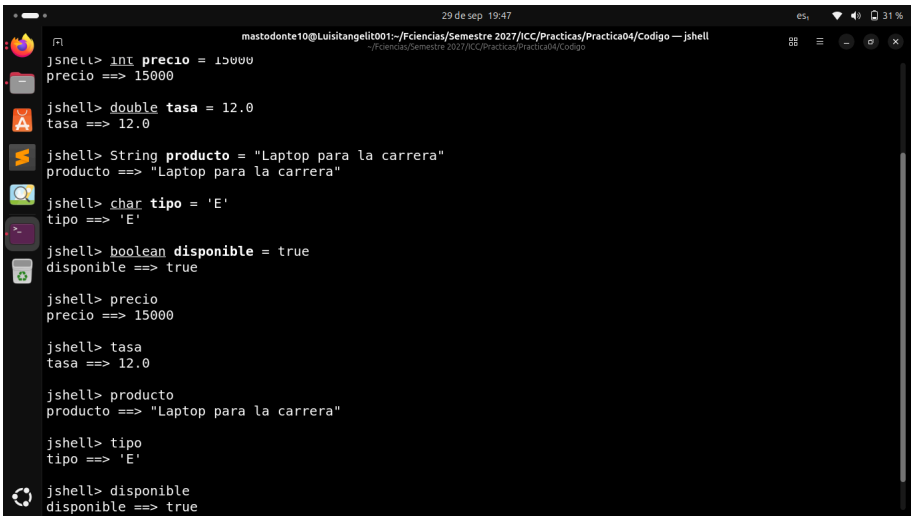
### Ejercicio 1: declaración de variables

Se declararon cinco variables, una por cada tipo de dato: int precio = 15000, double tasa = 12.0, String producto = "Laptop para la carrera", char tipo = 'E' y boolean disponible = true.




## 2.2 leer lo que hay en memoria

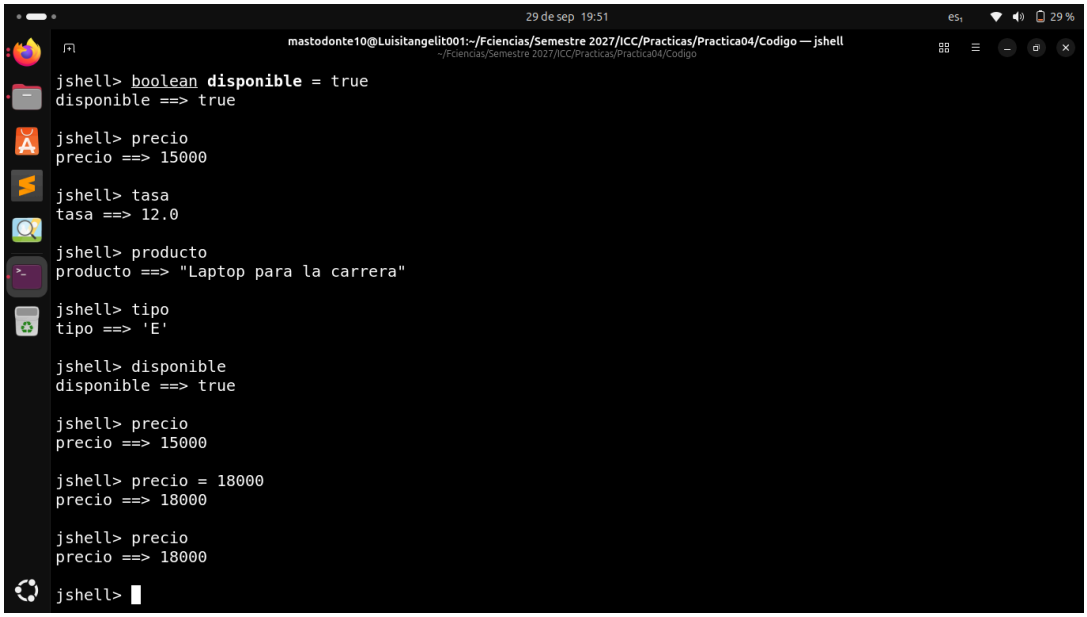
Al escribir únicamente el nombre de cada variable, JShell devolvió el valor guardado: 15000, 12.0, "Laptop para la carrera", 'E' y true. Es decir, se leyó el contenido de la memoria a través del nombre que se le dio.




## Ejercicio 3: las variables pueden variar

Se consultó precio (15000), luego se ejecutó precio = 18000 y se volvió a consultar. Se puede ver que el valor guardado cambió de **15000 a 18000**: la misma variable, con el mismo nombre y el mismo espacio de memoria, pasó a contener otro valor.

**Diferencia entre** int precio = 18000 **y** precio = 18000**:** el int *declara* la variable, es decir, le indica a Java el tipo (int), reserva el espacio y le pone nombre, y además le da un valor. La segunda sólo *asigna* un valor nuevo a una variable que ya teniamos.

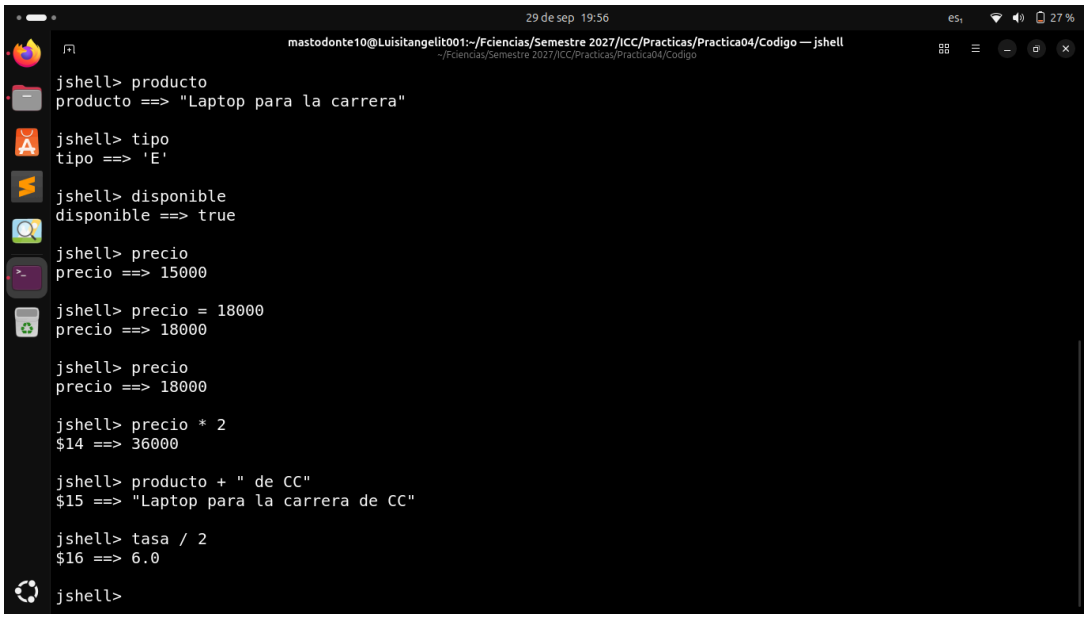



### Ejercicio 4: operaciones con variables

Se hicieron tres operaciones, cada una con una variable distinta:

- precio * 2 devolvió 36000. Es válida porque precio es un int y trabaja enteros
- producto + " de CC" devolvió "Laptop para la carrera de CC". Es válida porque el operador + entre String concatena (pega) texto.
- tasa / 2 devolvió 6.0. Es válida por ser un double el resultado conserva el decimal.

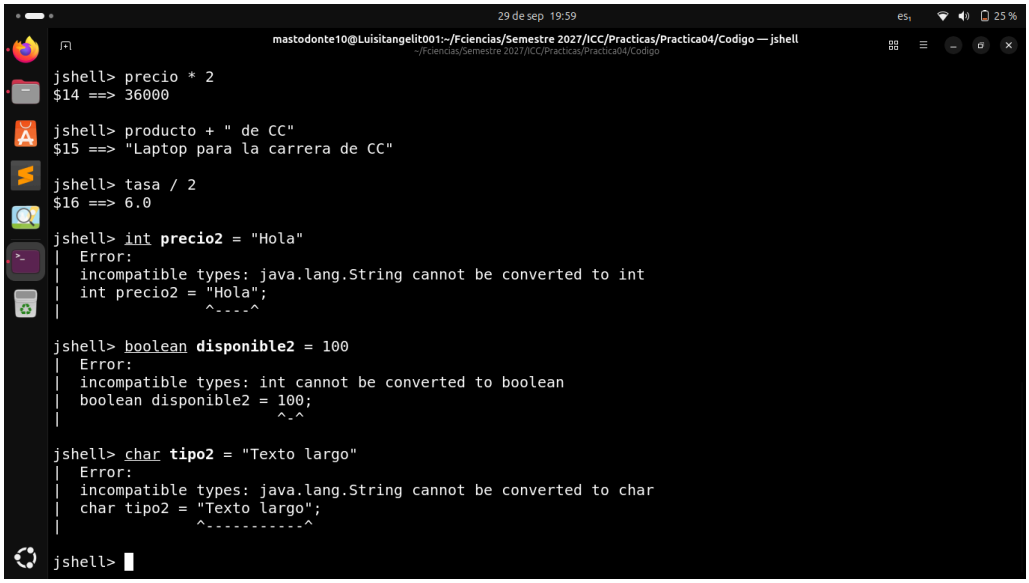
JShell guardó cada resultado con $ Las variables originales no se modificaron, porque las operaciones no se asignaron a ellas.



### Ejercicio 5: asignaciones que no corresponden con el tipo

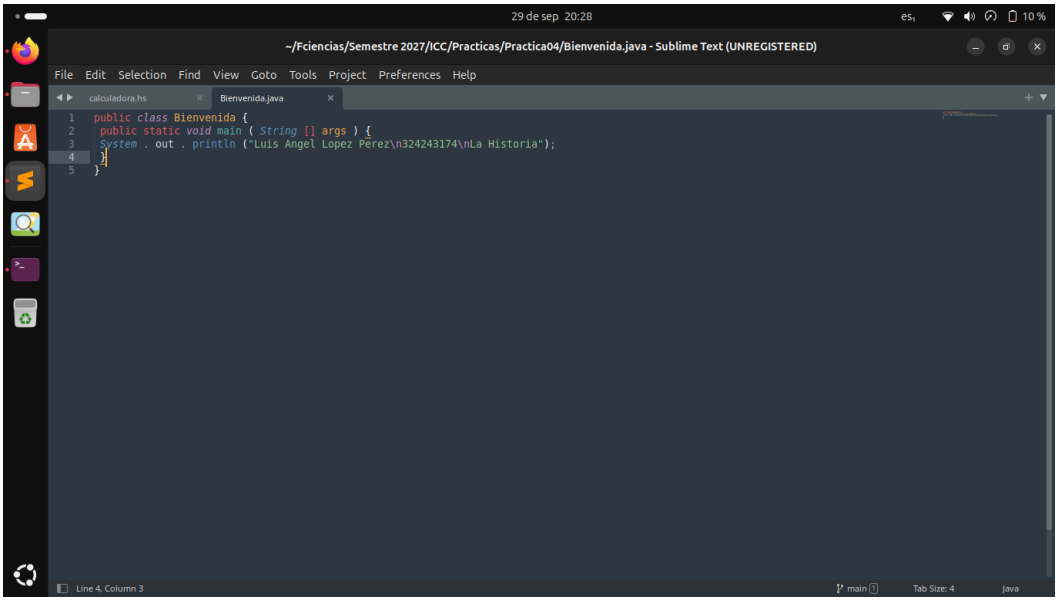
Se intentaron tres declaraciones con un valor que no corresponde al tipo de la variable. En las tres, JShell rechazó la declaración con un mensaje incompatible types:

| Declaración | Error del compilador | Qué entiendo de ese error |
|---|---|---|
| int precio2 = "Hola" | incompatible types: java.lang.String cannot be converted to int | Una variable int sólo guarda números enteros; "Hola" es texto. |
| boolean disponible2 = 100 | incompatible types: int cannot be converted to boolean | Un boolean sólo acepta true o false. |
| char tipo2 = "Texto largo" | incompatible types: java.lang.String cannot be converted to char | Un char guarda un solo carácter y se escribe con comillas simples ('E'). Con comillas dobles Java lo toma como String, y además el texto tiene varios caracteres. |



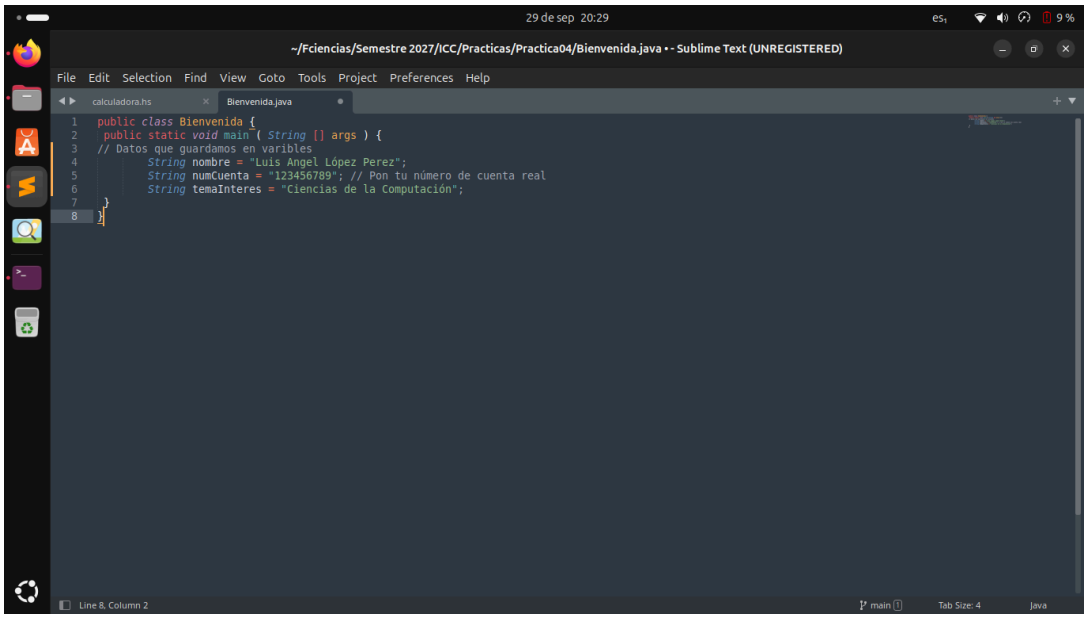

## 1.2 Agregando variables en la Práctica 2

Se copió Bienvenida.java de la Práctica 2 al directorio Practica04 y se abrió con Sublime Text. El código original imprimía los tres datos con una sola instrucción System.out.println que nos pidieron la otra vez.



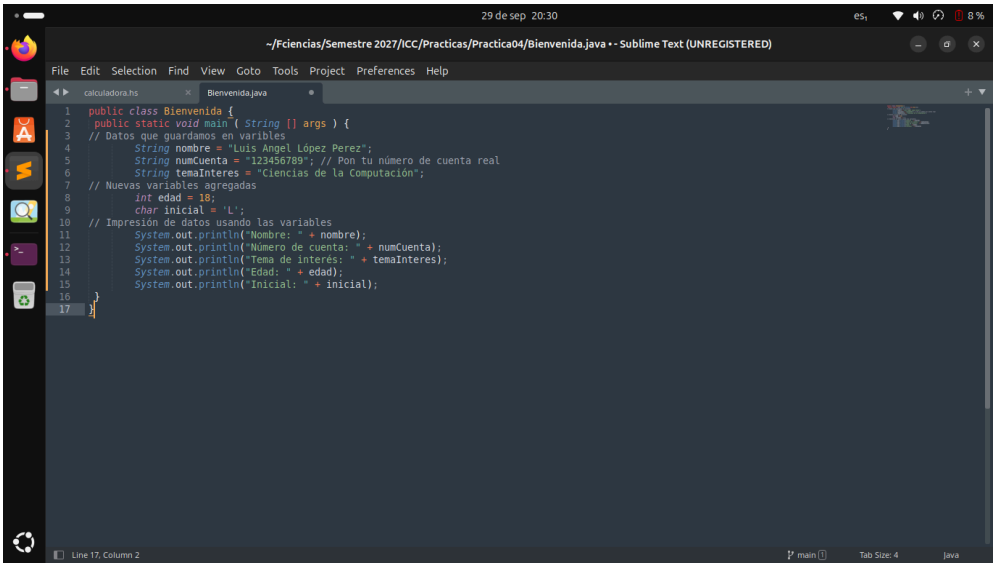

## Ejercicio 2: los tres datos en variables

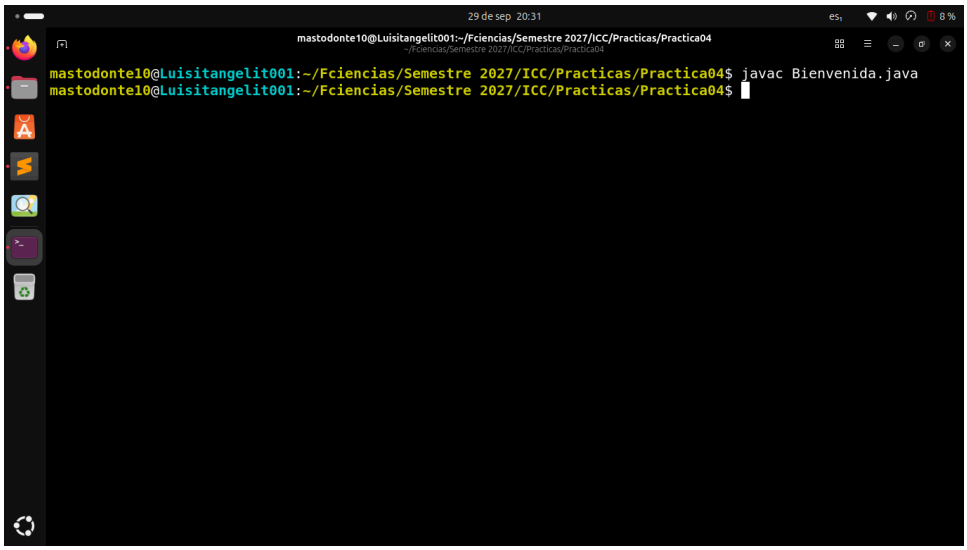
Se sacaron los tres datos del texto y se guardaron en tres variables String: nombre, numCuenta y temaInteres. Cada una quedó declarada dentro de main, con un comentario que explica el bloque.

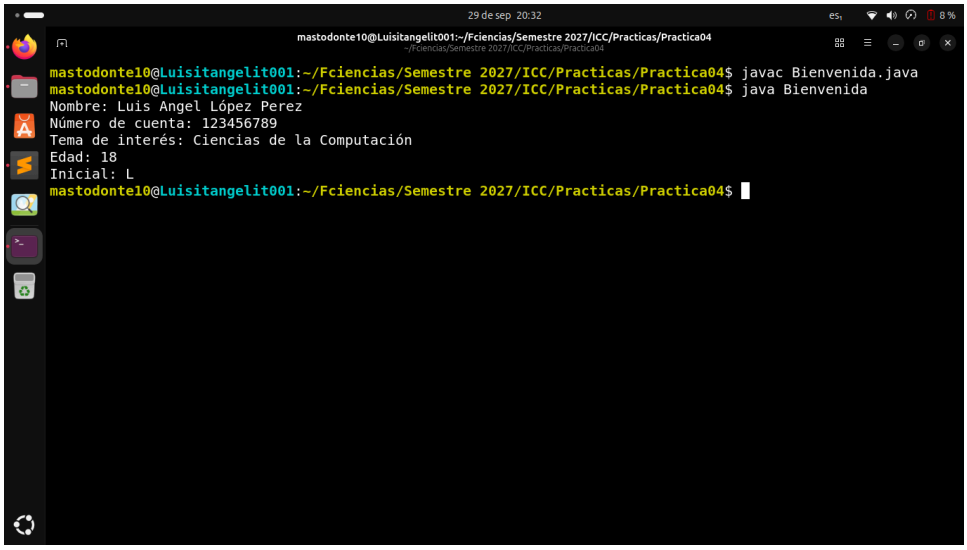



## Ejercicio 3: variables nuevas

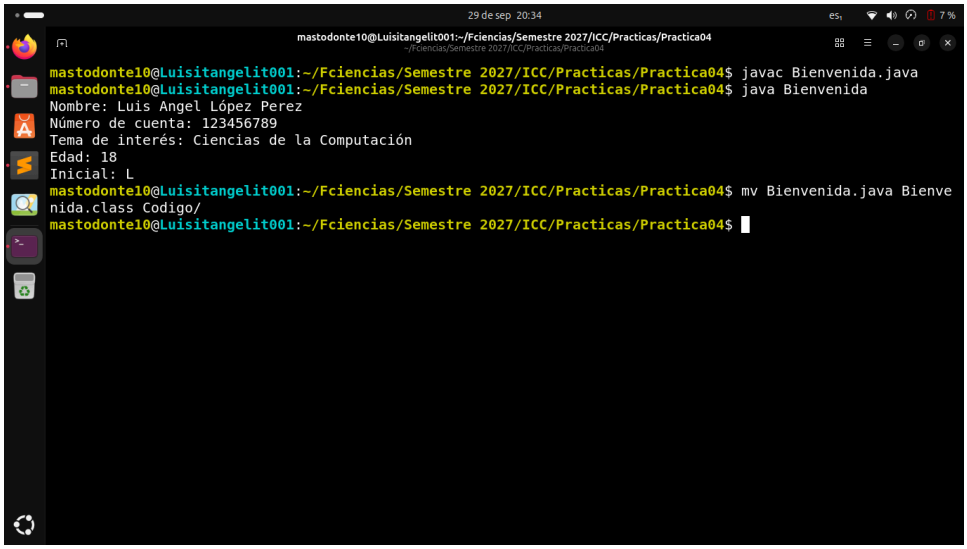
Se agregaron dos variables nuevas: int edad = 18 para el valor numérico y char inicial = 'L' para guardar un solo carácter. Después se reemplazó el println único por cinco instrucciones System.out.println, cada una con una etiqueta de texto concatenada con el valor de su variable ("Nombre: " + nombre, "Edad: " + edad, etc.). La salida muestra ahora las cinco líneas.




Al terminar, Bienvenida.java y Bienvenida.class se movieron al directorio Codigo con mv.

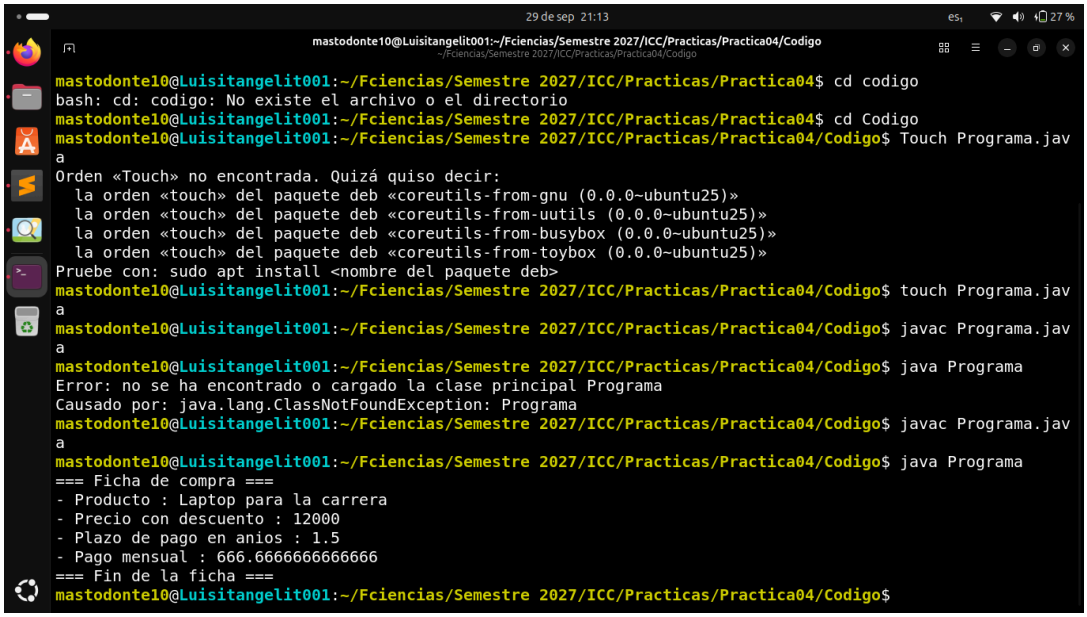



# Parte 2: Comentarios e impresiones elegantes

## 2.1 Comentarios en Java

Se descargó Programa.java de la tarea en Classroom, se copió a Practica04/Codigo y se agregó un comentario arriba de cada línea de código dentro de main. Después se guardó, se compiló y se ejecutó de nuevo para comprobar que la salida no cambiará.






En la terminal me sale un error de Programa.porque en ese momento el archivo no se había guardado en el editor, así que javac no generó la clase. Se guardó, se compiló otra vez y el programa se ejecutó bien. Con los comentarios la salida es idéntica a la de antes: los comentarios son texto que el compilador ignora por completo.

### Justificación de los comentarios

Escribí los comentarios pensando en alguien que no vio el enunciado y tiene que entender qué hace cada línea:

- **En las variables** expliqué por qué elegí cada tipo: String para el nombre del producto, int para precio y descuento (no hay laptops con centavos en este caso) y double para los meses, para que las divisiones den resultado con decimales.
- **En los println** indiqué qué dato se imprime y qué cálculo hay detrás: el precio con descuento es la resta, el plazo en años es meses entre 12.0 y el pago mensual es el precio con descuento entre los meses.
- Cada comentario va **arriba** de la línea que describe y dice el *para qué*, no sólo repite el código, de modo que un lector ajeno puede seguir el programa de arriba abajo sin ejecutarlo.

## 2.2 Printf

### Ejercicio 1: ¿qué es printf y cómo se usa?

Numerosos lenguajes de programación implementan una función **printf** (***print f**ormatted*), para mostrar una cadena con formato. Esta, originaria del lenguaje de programación C, tiene un prototipo similar al siguiente:

int printf(const char *formato, ...)

La cadena constante **formato** provee una descripción de la salida, con placeholders marcados por caracteres de escape "%", para especificar la localización relativa y el tipo de salida que la función debe producir.

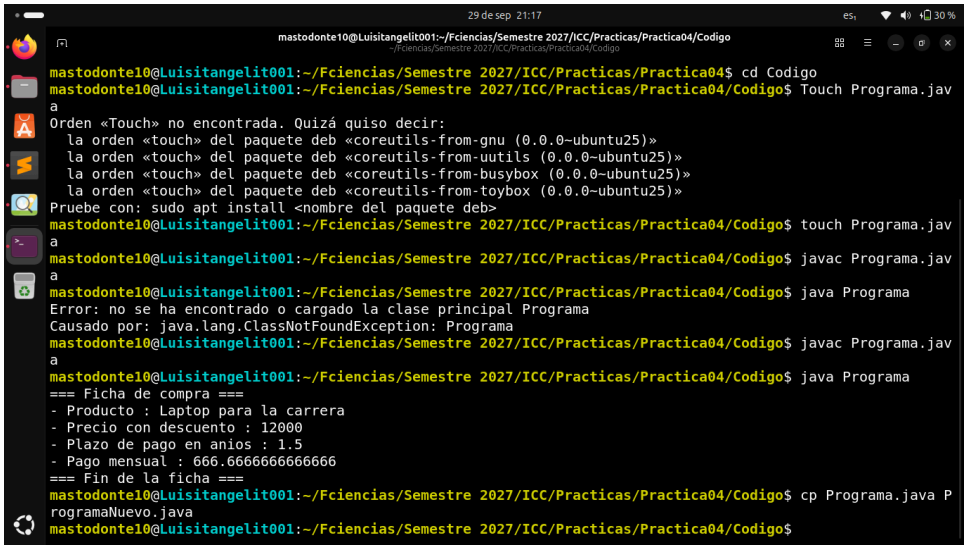
| Formateador | Salida |
|---|---|
| %d o %i | entero en base 10 con signo (int)<br>printf("El numero entero en base 10 es: %d", -10); |
| %u | entero en base 10 sin signo (int) |
| %o | entero en base 8 sin signo (int) |
| %x | entero en base 16, letras en minúscula (int) |
| %X | entero en base 16, letras en mayúscula (int) |
| %f | Coma flotante decimal de precisión simple (float) |
| %lf | Coma flotante decimal de precisión doble (double) |
| %ld | Entero de 32 bits (long) |
| %lu | Entero sin signo de 32 bits (unsigned long) |
| %e | La notación científica (mantisa / exponente), minúsculas (decimal precisión simple o doble) |
| %E | La notación científica (mantisa / exponente), mayúsculas (decimal precisión simple o doble) |
| %c | carácter (char) |
| %s | cadena de caracteres (string) |

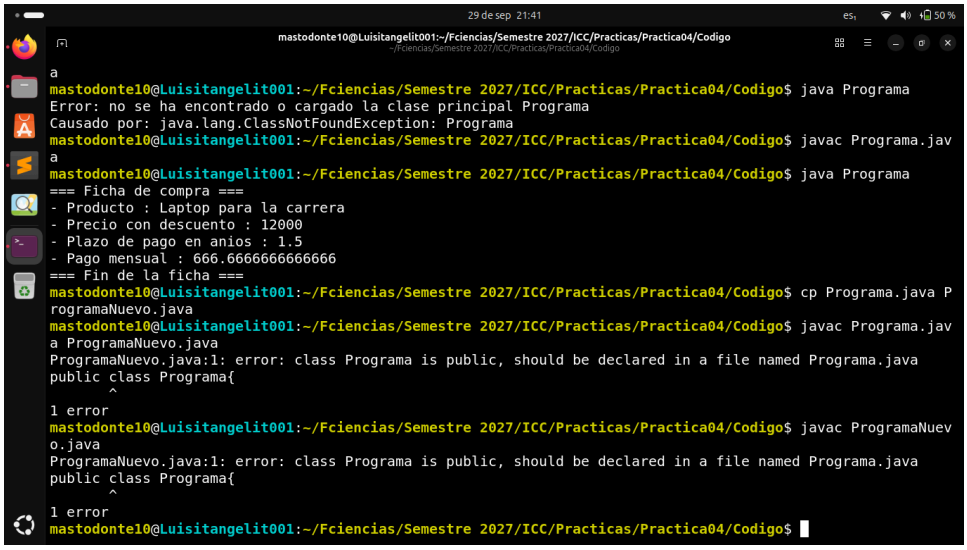
Fuente:

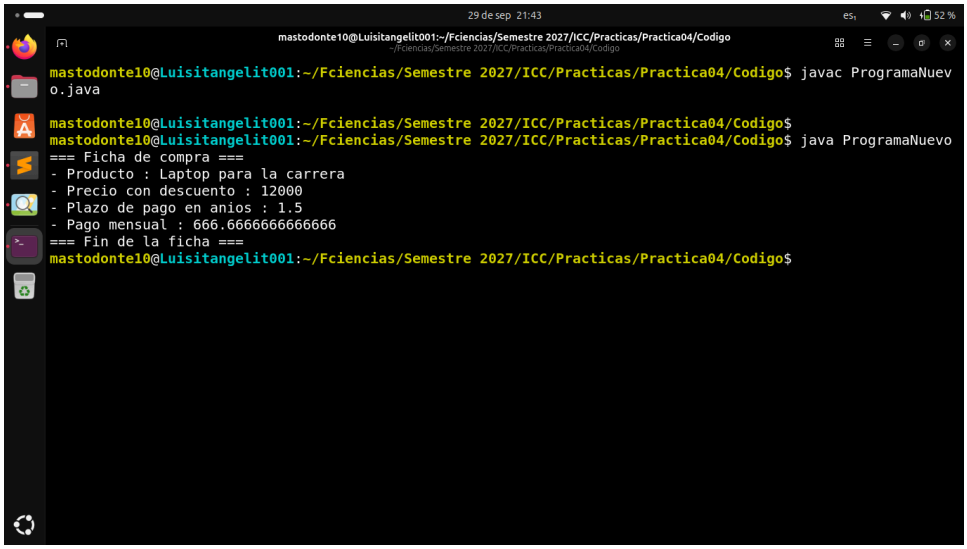
**colaboradores de Wikipedia. (2023, 1 mayo).** ***Printf*****.** https://es.wikipedia.org/wiki/Printf

## Ejercicio 2: copia ProgramaNuevo.java

Se hizo una copia de Programa.java con cp Programa.java ProgramaNuevo.java. Al compilarla, el compilador marcó un error: class Programa is public, should be declared in a file named Programa.java. Esto pasó porque dentro del archivo la clase todavía se llamaba Programa, y en Java una clase pública debe llamarse igual que el archivo que la contiene. Se cambió la primera línea a public class ProgramaNuevo, se guardó y el archivo compiló y se ejecutó sin problemas.






### Ejercicios 3 y 4: pago mensual con printf y un solo printf

Con printf usé el formato %.2f, que redondea el número a 2 decimales, como un precio real. El %n agrega el salto de línea y el $ va como texto fijo. Ahora la salida es $666.67. Adjunta captura del código

Junté los cuatro println en un solo printf. El primer argumento es el texto con cuatro marcadores y, después de la coma, van los valores en el mismo orden: %s recibe producto, %d recibe el precio con descuento (un int), %.1f recibe el plazo en años y %.2f recibe el pago mensual. Cada línea termina con %n. (no se tomaron las screenshots :[)

# Parte 3: Reto de abstracción

## 3.1 Filtrando información

Después de leer el problema completo se separaron los datos en dos conjuntos: los necesarios para programar lo que pide el señor Pines (intereses, total a pagar y mensualidad de Robbie) y los innecesarios.

### Datos que ocupamos

| Dato | Valor | Por qué sí |
|---|---|---|
| Precio de la laptop (capital) | $12,899 | Es el dinero que se presta; entra directo en la fórmula del interés y en el total. |
| Tasa de interés anual | 15 % | Es el porcentaje que se cobra por año; es el segundo dato de la fórmula. |
| Plazo de pago | 21 meses | Se convierte a años para el interés y sirve para dividir el total en mensualidades. |
| Clasificación del cliente | 'E' (estándar) | No cambia el cálculo, pero el señor Stan pidió que quedara anotada desde el principio por si el programa crece. |
| Nombre del cliente | Robbie Valentino | No interviene en el cálculo, pero identifica a quién pertenece la cotización en la salida. |

### Datos inútiles

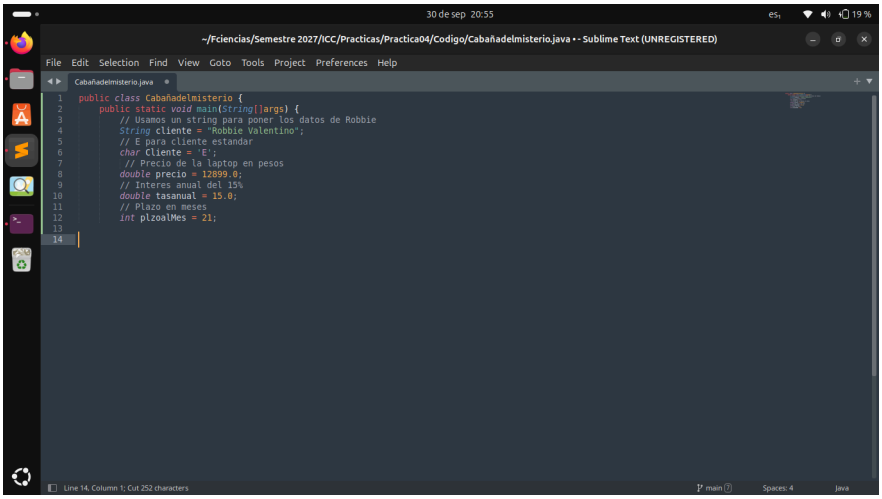
| Dato | Por qué no |
|---|---|
| Stanley Pines, treinta años atendiendo el museo, nombre de "La cabaña del misterio", tienda de regalos, Gravity Falls | Es el contexto de la historia; ningún valor entra en los cálculos. |
| Dipper, Mabel y Pato | Son personajes y distracciones de la narración que no aportan números. |
| Características de la laptop: 16 GB de RAM, SSD de 512 GB, ocho núcleos, 1.8 kg, colores gris o azul marino | Describen el producto, pero el préstamo sólo depende de cuánto cuesta, no de cómo es. |
| Rango de precios del pueblo ($11,249 a $14,479) | Es información de la investigación del Dipper pero el precio del Robbie ya viene dado. |
| Que los clientes sean alumnos de ICC y que haya una oportunidad de negocio | Es la motivación del tío Stan, no un dato de entrada. |

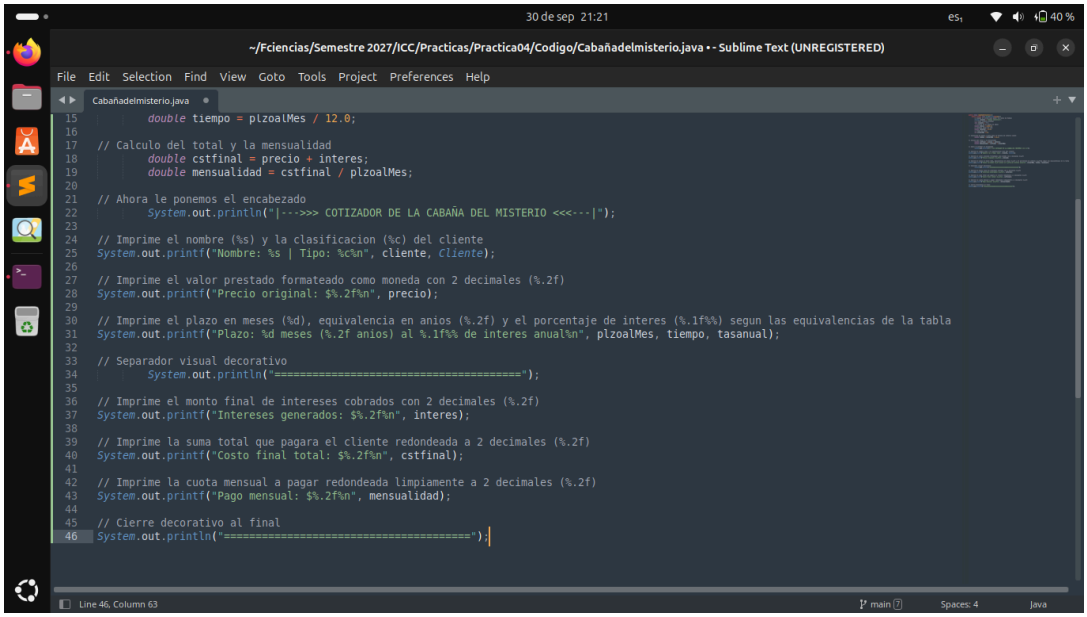
### Dato que más trabajo costó clasificar

Creo que las características de la laptop ya que pense que se tenían que clasificar por mi poca comprensión lectora.

## 3.2 Cumpliendo los deseos del señor Pines

Se creo el archivo empezó con Cabañadelmisterio.java porque no vi que sí o sí era Cotizador.java(luego lo corregi).






## Decisiones de diseño

- **Tipos de variable.** cliente es String (texto), la clasificación es char (un solo carácter, 'E'), el precio y la tasa anual son double (admiten decimales) y el plazo en meses es int, porque son meses completos.
- **Conversión de meses a años.** tiempo = plzoalMes / 12.0. Dividí entre 12.0 y no entre 12 para que Java haga la división con decimales; con enteros, 21 / 12 daría 1 y el interés saldría mal. Con 21 meses el tiempo es 1.75 años.
- **Cálculos.** El interés simple usa la fórmula capital × tasa anual / 100 × tiempo en años. El costo final es el precio más los intereses y la mensualidad es el costo final entre el plazo en meses. Cada paso se guardó en una variable, para no repetir cuentas.
- **Salida con printf.** Usé %s para el nombre, %c para la clasificación, %d para los meses, %.2f para los montos en pesos, %.1f%% para el porcentaje (el %% imprime el símbolo %) y %n para el salto de línea. Cada dato lleva una etiqueta (Precio original, Intereses generados, Costo final total, Pago mensual) para que no haya dudas al leer la terminal.
- **Comentarios.** Cada bloque lleva un comentario arriba que indica qué calcula o qué imprime y qué significa su formato, para que el ayudante del laboratorio pueda seguir la implementación.

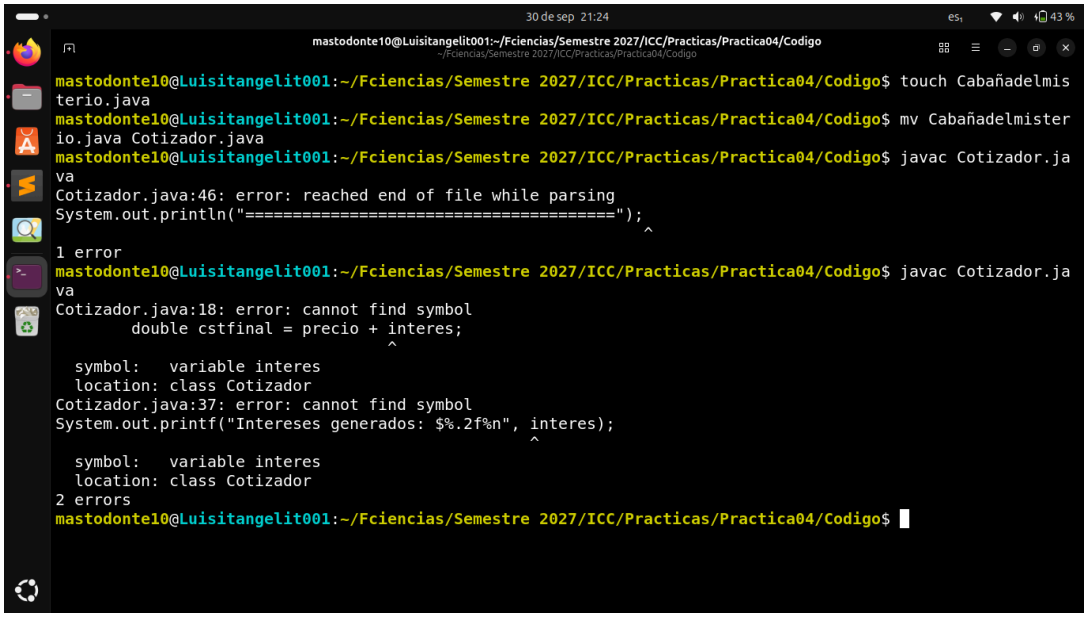
## Errores que aparecieron al compilar

Al compilar Cotizador.java aparecieron dos tipos de error, que corregí uno por uno:

- reached end of file while parsing: el archivo terminaba sin cerrar las llaves del método y de la clase. Al agregar las llaves que faltaban, el error desapareció.
- cannot find symbol: variable interes (líneas 18 y 37): usaba interes en el costo final y en el printf sin haberla declarado. Se declaró la variable con la fórmula del interés simple y la compilación terminó sin errores.

```
// Conversion de meses a anios para la formula de interes simple
        double tiempo = plzoalMes / 12.0;

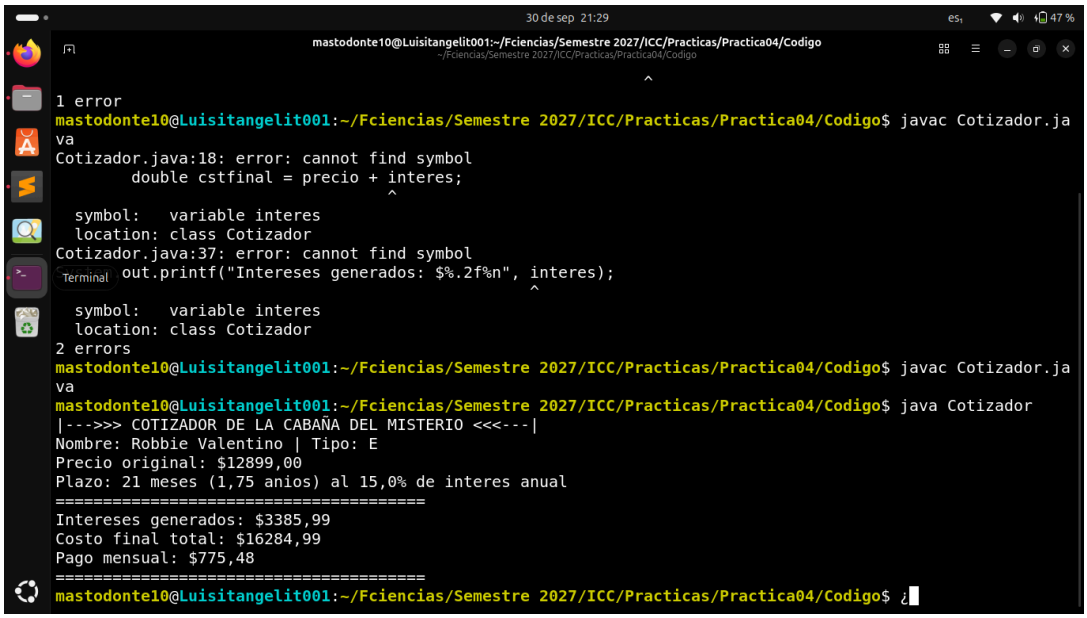
// Para el interes
        double interes = precio * (tasanual / 100.0) * tiempo;
```




## Compilación y ejecución

Con el código corregido, javac Cotizador.java compiló sin errores y java Cotizador imprimió la cotización de Robbie: precio original $12,899.00, plazo de 21 meses (1.75 años) al 15.0 % anual, **intereses de $3,385.99, costo final de $16,284.99 y pago mensual de $775.48**.

Revisé los resultados a mano: 12,899 × 15 / 100 × 1.75 = 3,385.99; 12,899 + 3,385.99 = 16,284.99; y 16,284.99 / 21 ≈ 775.48. Los tres coinciden con la terminal. Un detalle que noté es que la salida usa coma decimal ($775,48), porque printf toma el formato numérico de la configuración regional del sistema, que está en español.



## Conclusiones

Esta práctica me ayudó a entender qué significa darle memoria a un programa: declarar una variable es apartar un espacio con nombre y con tipo, y el tipo decide qué se puede guardar ahí. JShell me lo mostró con los errores de tipos incompatibles, y en el Cotizador lo comprobé de otra forma: dividir entre `12.0` y no entre `12` cambia el resultado. También vi que `printf` sirve para mostrar los montos con dos decimales y en un formato que sí parece un precio.

Los tropiezos fueron varios y todos dejaron algo: escribir `Touch` y `cd codigo` con la mayúscula equivocada (la terminal distingue entre mayúsculas y minúsculas), compilar un archivo sin haberlo guardado y recibir `ClassNotFoundException`, dejar el nombre de la clase anterior en `ProgramaNuevo`, olvidar cerrar llaves y usar una variable sin declararla. La lección fue leer completo el mensaje del compilador, porque casi siempre indica el archivo, la línea y la causa. Por último, el reto de abstracción me hizo ver que, antes de escribir código, hay que decidir qué datos del problema realmente se usan, porque la mayor parte de la historia del señor Pines no hacía falta para programar.